
Does More Exploration Actually Help? Comparing Exploration Strategies in DQN

Samuel Kostiuk Max Bullach Daniel Gleim
<https://github.com/samuelkos17/rl-project>

1 Motivation

DQN needs some way to explore the environment, and there are several common options such as ϵ -greedy, Boltzmann exploration, count-based bonuses, or NoisyNets, but it is not obvious how much the choice of exploration strategy actually matters, or why one strategy would work better than another. We want to compare a few strategies and also try to measure *why* some of them work better, not just which one wins.

2 Related Topics

This connects to the lecture topics on Q-Learning/DQN and the exploration-exploitation trade-off. The strategies we compare are based on ideas from the lecture and a few papers: count-based exploration bonuses [1], NoisyNets as a learned alternative to ϵ -greedy[2], and the `rliable` paper on how to properly compare RL results with confidence intervals instead of single runs[3].

3 Idea

We take one DQN implementation and only change how it explores, so we can compare strategies fairly. We plan to test ϵ -greedy (our baseline), Boltzmann exploration, a count-based bonus, and NoisyNets, on a few MiniGrid environments that get progressively harder to explore.

Besides just comparing final performance, we also want to log how much of the environment each strategy actually visits during training (state coverage). Our idea is to check whether strategies that explore more broadly early on also end up performing better later, especially in the harder, more sparse environments. This gives us not just a ranking of strategies, but also a possible explanation for why some work better.

Algorithm 1 DQN with a swappable exploration strategy.

Require: environment e , exploration strategy X **return** trained policy π
 while T **do** RUE
 Choose action using strategy X
 Take a step in e , store the transition
 Update DQN on a batch of stored transitions
 Log evaluation return and visited states
 end while

4 Experiments

Environments & Metrics We plan to use three MiniGrid environments of increasing exploration difficulty: an easy one as a sanity check (Empty), a medium one that needs a few steps in the right

order (DoorKey), and a harder, more sparse one (MultiRoom). We will measure evaluation return over training, and additionally track how much of the state space each strategy visits over time.

Experimental Scope We will run each of the four strategies on each environment with multiple seeds, so we can report results with confidence intervals instead of single lucky/unlucky runs.

Estimated Computational Load The environments we chose are small, so individual runs should only take somewhere between 20-40 minutes on a single GPU. Given the number of strategies, environments, and seeds, we expect a total of a few GPU-hours, which we consider feasible within our timeline, also since runs can be done in parallel.

5 Timeline

The proposal is due on 13.07 and the report is due on 23.08, so we have about six weeks. We plan roughly as follows:

- **Research:** read up on the exploration strategies and the evaluation methodology we want to use.
- **Implementation:** set up DQN and implement the four exploration strategies plus the coverage logging, split up between the three of us.
- **Experiments:** run the main comparison first, then the smaller hyperparameter sweep. We want to start this early to leave room for fixing bugs.
- **Analysis:** aggregate results, make plots, and check whether early exploration coverage predicts later performance.
- **Reporting:** write the report, clean up the code, and prepare the poster.

References

- [1] M. G. Bellemare et al. Unifying count-based exploration and intrinsic motivation. In *NeurIPS*, 2016.
- [2] M. Fortunato et al. Noisy networks for exploration. In *ICLR*, 2018.
- [3] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *NeurIPS*, 2021.